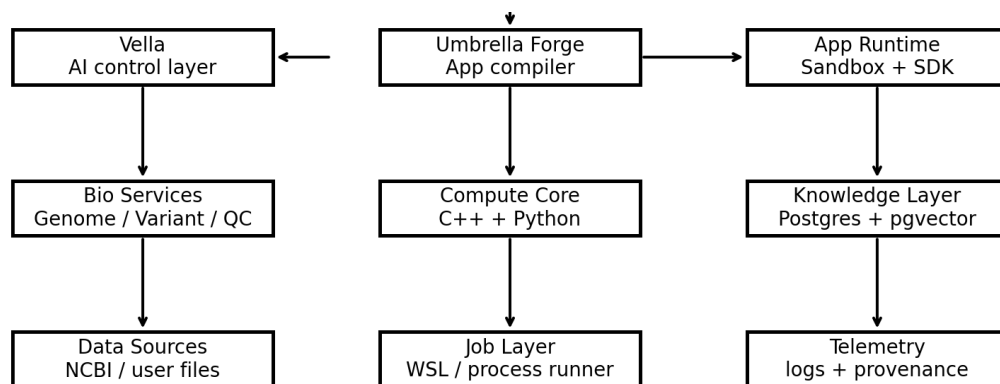


UMBRELLA OS

Backend + Machine Learning Build Guide

18.5-hour hackathon implementation plan → full scientific platform architecture



All scientific actions produce structured provenance and auditable results.

Document purpose

This document is the implementation-level backend guide for Umbrella OS. It describes the minimum viable backend, how to acquire and stage a large bacterial genome/AMR corpus, how to run AMRFinderPlus, how to construct labels from laboratory phenotype records, how to train reproducible ML models from sequence-derived features, and how to expose the resulting computation to the Umbrella desktop applications.

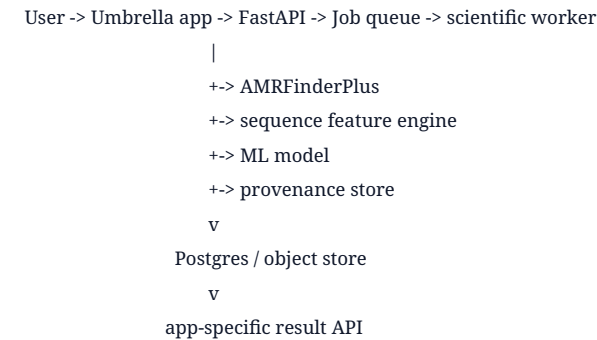
Important framing: reserve at least 120 GB of local/server storage for the challenge corpus and derived artifacts. Do not assume AMRFinderPlus itself is a 120 GB database; NCBI publishes AMRFinderPlus software and its curated reference resources separately. The 120 GB target should be treated as the minimum data-lake capacity for genomes, metadata, annotations, intermediate files, indexes, models, logs and cached outputs unless the organizer-pinned benchmark specifies a different footprint. [1][2]

1. Executive build target

Umbrella OS should have a thin desktop, a strong computation backend, and a domain-specific application layer. The backend is the source of truth: every sample, annotation, model version, simulation result and provenance record should be addressable and reproducible.

Layer	Responsibility	MVP implementation
UI / OS	Desktop, apps, windows, terminal	Next.js 15 + React + Tailwind
AI control	Intent parsing, workflow planning, explanation	FastAPI tool router + LLM API
API	Stable contracts for apps	FastAPI + Pydantic
Scientific core	FASTA/QC, k-mers, mutation checks, scoring	Python + C++ core; pybind11 or subprocess
AMR engine	Known AMR genes / mutations	AMRFinderPlus
Data	Metadata, phenotype, results	PostgreSQL + S3/MinIO
Jobs	Long-running analyses	Redis + Celery/RQ
ML	Classification/calibration	scikit-learn; optional PyTorch
Ops	Containers, logs, metrics	Docker + OpenTelemetry/Prometheus

The critical backend flow



Definition of done for the hackathon

- A sample FASTA can be uploaded, analyzed, and tracked by job ID.
- AMRFinderPlus runs against a pinned database version and stores version metadata.
- Laboratory phenotype records are joined by stable genome/sample identifiers.
- At least one antibiotic has a trained baseline classifier with a held-out test set and calibrated probability.
- Genome, Mutation, AMR and Analysis apps consume the same backend contracts.
- Every result contains provenance: input hash, software version, database version, model version, timestamps.

2. Phase map: build in this order

Phase	Time	Build	Outcome
0	0:00-0:30	Environment + repo	Running monorepo
1	0:30-2:00	Data landing + metadata	Challenge data visible
2	2:00-4:00	FastAPI + DB + object storage	Upload/job API
3	4:00-6:30	Sequence engine + QC	FASTQ/FASTA analysis
4	6:30-9:00	AMRFinderPlus pipeline	AMR annotation output
5	9:00-12:00	Feature + label pipeline	Training matrix
6	12:00-14:00	ML + calibration	Working model
7	14:00-16:00	Apps + desktop integration	4 core apps
8	16:00-17:30	Vella + Forge stub	Natural-language workflows
9	17:30-18:30	Demo data + tests	Stable showcase
10	18:30-18:30+	Buffer	No new features

Rule: separate “critical path” from “stretch”

Critical	Stretch
FASTAPI service	Full distributed execution
Postgres + local object store	AWS S3 + RDS deployment
AMRFinderPlus	ResFinder + cAMRah cross-check
Marker-feature logistic model	DNABERT-2 embeddings
Genome + Mutation + AMR apps	Pathogen Atlas + Research Desk
Small simulated DNA stress visualizer	Full multi-factor simulator
Prebuilt App Forge template	Arbitrary code generation

The fastest route to a trustworthy model is not to train a language model from raw genomes during the sprint. Use the supplied laboratory phenotype data as the label source, derive a compact interpretable feature matrix from AMRFinderPlus plus sequence features, train a CPU baseline, and make deep learning an optional comparison track. This is consistent with the challenge guidance you provided and with the role of AMRFinderPlus as a curated AMR detector. [1][3]

3. Repository and local environment

Use WSL2 Ubuntu as the primary build environment. Keep the web UI and backend in one monorepo but isolate scientific workers from the browser-facing process. Do not run heavy genome jobs inside Next.js server actions.

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y git curl build-essential cmake pkg-config \
libssl-dev libcurl4-openssl-dev libhdf5-dev \
python3 python3-venv python3-pip jq pigz lftp
```

```
# Node / pnpm
corepack enable
pnpm --version
```

```
# Python environment
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install fastapi uvicorn[standard] pydantic-settings \
sqlalchemy psycopy[binary] alembic redis celery \
pandas numpy scipy scikit-learn joblib pyarrow \
biopython psutil python-multipart
```

Recommended directory layout

```
umbrella-os/
apps/           # Next.js desktop + native apps
services/api/   # FastAPI
workers/science/ # long-running sequence jobs
engine/         # C/C++ algorithms
ml/            # feature building + training
pipelines/      # ETL/annotation jobs
data/          # NOT committed to Git
raw/
metadata/
annotations/
features/
labels/
indexes/
models/
cache/
db/
migrations/
infra/
.kiro/         # specs, steering, hooks
```

Environment variables

```
DATABASE_URL=postgresql+psycopy://umbrella:umbrella@localhost:5432/umbrella
REDIS_URL=redis://localhost:6379/0
OBJECT_STORE_ENDPOINT=http://localhost:9000
OBJECT_STORE_BUCKET=umbrella-data
AMRFINDER_DB=/opt/amrfinder/data/latest
MODEL_DIR=/workspace/data/models
MAX_UPLOAD_GB=10
```

4. Compute and storage plan

Treat the 120 GB requirement as a floor for the local data workspace, not as the size of one special database file. Whole-genome FASTA, protein files, annotations, extracted features, intermediate outputs, model artifacts and indexes can multiply storage usage. Plan for 1.5-2x temporary overhead when rebuilding indexes or running bulk pipelines.

Resource	Minimum	Preferred
CPU	8 cores	16+ cores
RAM	16 GB	32-64 GB
SSD free	120 GB	250-500 GB
GPU	Optional	1 NVIDIA GPU for embeddings
Network	Stable broadband	High sustained throughput
OS	WSL2 Ubuntu	Native Linux/WSL2

Storage allocation

Area	Target
Raw genomes	60-80 GB
Metadata + phenotype	1-5 GB
AMRFinderPlus database	versioned, usually much smaller than corpus
Annotations / reports	10-20 GB
Feature matrices / Parquet	5-20 GB
Indexes / k-mer DBs	10-40 GB
Models / experiment logs	2-10 GB
Temporary workspace	20-40 GB reserve

Use Parquet for wide feature tables and compressed TSV/CSV only for interchange. Keep raw files immutable. Generate all derived artifacts into versioned folders. Never overwrite a production model or raw benchmark in place.

Docker local stack

```
docker compose up -d postgres redis minio

# optional health checks
docker compose ps
curl http://localhost:9000/minio/health/live
```

5. Acquire the genome + phenotype benchmark

BV-BRC is the primary source in the challenge brief you supplied. Its current documentation exposes genome, contig, feature, and AMR data through APIs and command-line tools; its FTP site now requires explicit FTPS for bulk distribution. [1][4]

Start from the organizer-pinned dataset

The most reliable sequence is: take the organizer-provided list of genome IDs and phenotype rows first, then fetch only the matching genome files. This prevents accidental label drift, avoids wasting bandwidth, and makes your benchmark reproducible.

```
mkdir -p data/raw/genomes data/metadata
```

```
# Example: fetch release metadata index
```

```
wget -qN https://ftp.bv-brc.org/RELEASE_NOTES/genome_summary \
-O data/metadata/genome_summary
```

```
wget -qN https://ftp.bv-brc.org/RELEASE_NOTES/genome_metadata \
-O data/metadata/genome_metadata
```

```
# Organizer-provided genome_list.txt contains one stable BV-BRC genome_id per line.
```

```
cat data/metadata/genome_list.txt | head
```

FTPS bulk download

BV-BRC documents explicit FTPS access and provides an example using wget. The path layout is organized by genome ID and exposes .fna, .faa, feature tables, GFF, and specialty-gene files. [4]

```
while read -r id; do
  mkdir -p "data/raw/genomes/$id"
  wget -qN "https://ftp.bv-brc.org/genomes/$id/$id.fna" \
  -O "data/raw/genomes/$id/$id.fna"
done < data/metadata/genome_list.txt
```

Verification

```
find data/raw/genomes -name '*.fna' | wc -l
sha256sum data/raw/genomes/*/*.fna > data/metadata/fna.sha256
```

For the challenge, do not silently substitute Kaggle mirrors or inferred phenotype fields. The organizer brief explicitly says to use the pinned laboratory-measured test results. BV-BRC exposes evidence type, laboratory method, testing standard, measurement, and phenotype fields, which lets you retain provenance rather than collapsing everything into an opaque label. [3]

6. Build the 120 GB data lake

Use a manifest-driven download system so you can resume, audit and reproduce the exact corpus. Keep a single source-of-truth manifest with genome ID, source URL, local path, file type, byte size, SHA-256 and acquisition timestamp.

```
id,source_url,local_path,file_type,bytes,sha256,downloaded_at
511145.12,https://ftp.bv-brc.org/genomes/511145.12/511145.12.fna,data/raw/genomes/511145.12/511145.12.fna,fna,0,,
```

Python manifest writer

```
from pathlib import Path
import hashlib, csv, time

def sha256(path, chunk=8<<20):
    h = hashlib.sha256()
    with open(path, 'rb') as f:
        while b := f.read(chunk): h.update(b)
    return h.hexdigest()

root = Path('data/raw/genomes')
rows=[]
for p in root.rglob('*.fna'):
    rows.append({
        'local_path': str(p),
        'file_type': 'fna',
        'bytes': p.stat().st_size,
        'sha256': sha256(p),
        'downloaded_at': int(time.time()),
    })
with open('data/metadata/files.csv','w',newline='') as f:
    w=csv.DictWriter(f, fieldnames=rows[0].keys()); w.writeheader(); w.writerows(rows)
```

Data layout

```
data/
raw/genomes/<genome_id>/*.fna
raw/proteins/<genome_id>/*.faa
metadata/genome_summary
metadata/genome_metadata
metadata/amr_lab.tsv
annotations/amrfinder/<db_version>/<genome_id>.tsv
features/amr/<version>.parquet
features/sequence/<version>.parquet
labels/<breakpoint_or_pinned_label_version>.parquet
splits/<split_id>.json
models/<antibiotic>/<model_version>/
reports/<run_id>/
cache/
```

The data-lake boundary matters: the browser should never need to know where 120 GB of genomes are. It requests a sample or analysis job by ID, and the backend returns a result object or streamable artifact.

7. Normalize BV-BRC laboratory AMR records

The AMR table is the label spine. Current BV-BRC API documentation includes antibiotic name, evidence, laboratory typing method, measurement, measurement sign/value/unit, resistant phenotype, testing standard and year, genome ID, and provenance fields. [3]

Canonical field	Use in Umbrella
genome_id	Primary join key to genome
antibiotic	Target variable / model name
evidence	Must be Phenotype for label training unless brief says otherwise
measurement	Raw lab observation retained
measurement_sign	Needed for censored MIC values
measurement_value	Numeric ML input for optional MIC model
measurement_unit	Interpretation
testing_standard + year	Breakpoint provenance
laboratory_typing_method	Method stratification / QC
resistant_phenotype	Convenient label if explicitly lab-derived
source	Provenance

Normalization rules

- Canonicalize antibiotic names with an explicit mapping file; never fuzzy-map a drug label in code without logging the mapping.
- Retain raw measurement strings and all provenance columns in the normalized table.
- Use only rows marked as laboratory phenotype for the challenge model unless the organizer explicitly instructs otherwise.
- Exclude or separately model intermediate/non-susceptible categories rather than silently calling them susceptible.
- If training from MIC, use the organizer-pinned breakpoint rules or standardized labels for the challenge. Do not invent breakpoints.

```
CREATE TABLE amr_lab (
  record_id TEXT PRIMARY KEY,
  genome_id TEXT NOT NULL,
  antibiotic TEXT NOT NULL,
  evidence TEXT NOT NULL,
  measurement TEXT,
  measurement_sign TEXT,
  measurement_value DOUBLE PRECISION,
  measurement_unit TEXT,
  resistant_phenotype TEXT,
  testing_standard TEXT,
  testing_standard_year INT,
  laboratory_typing_method TEXT,
  source TEXT,
  raw_json JSONB
);
```

8. Install AMRFinderPlus and pin versions

NCBI states that AMRFinderPlus identifies AMR genes, resistance-associated point mutations and selected additional gene classes from protein annotations and/or assembled nucleotide sequence. The software and databases are open source and the database is versioned independently; record both software and database versions for reproducibility. [2][5]

Preferred installation path

```
# option A: conda / bioconda if available in your environment
conda create -n umbrella-amr -c conda-forge -c bioconda \
  ncbi-amrfinderplus
conda activate umbrella-amr

amrfinder --version
amrfinder_update -h
```

Database update and version pinning

```
mkdir -p $HOME/.local/share/amrfinderplus/data
amrfinder_update -d $HOME/.local/share/amrfinderplus/data

# inspect the database version
amrfinder --database_version
```

The NCBI updater creates a versioned database directory and can maintain a latest link. Pin the exact directory for the challenge run rather than pointing at a moving “latest” symlink in a training manifest. [5]

Run nucleotide screening

```
DB=$HOME/.local/share/amrfinderplus/data/latest

amrfinder -n sample.fna -O Escherichia --plus \
  --database "$DB" \
  --output sample.amrfinder.tsv
```

For bulk analysis, cache the output keyed by input SHA-256 + AMRFinderPlus software version + database version + organism mode. This prevents accidental reuse of stale annotations after a database update.

9. Bulk AMRFinderPlus worker

Run one process per genome as a queued worker. Keep concurrency below the practical CPU/RAM limit; the goal is throughput with reproducibility, not maximum process count.

```
#!/usr/bin/env bash
set -euo pipefail
ID="$1"
FA="$2"
DB="$3"
OUT="data/annotations/amrfinder/${ID}.tsv"
mkdir -p "$(dirname "$OUT")"

if [[ -s "$OUT" ]]; then
    echo "cached: $ID"
    exit 0
fi

amrfinder -n "$FA" --plus --database "$DB" \
    --output "$OUT"

echo "$ID,$(sha256sum "$FA" | cut -d ' ' -f1),$(amrfinder --database_version)" \
    >> data/annotations/amrfinder/manifest.csv
```

Worker orchestration

```
from celery import Celery
from pathlib import Path

app = Celery('umbrella', broker='redis://localhost:6379/0')

@app.task
def amr_job(genome_id, fasta_path, db_path):
    # Call the pinned AMRFinderPlus binary in a controlled subprocess.
    # Return only an artifact path + versions; large TSVs stay on disk/object storage.
    import subprocess, os
    out = f'data/annotations/amrfinder/{genome_id}.tsv'
    Path(out).parent.mkdir(parents=True, exist_ok=True)
    subprocess.run([
        'amrfinder', '-n', fasta_path, '--plus', '--database', db_path,
        '--output', out
    ], check=True)
    return {'genome_id': genome_id, 'artifact': out}
```

Why not call AMRFinderPlus from the browser?

Because a browser request should complete in milliseconds to seconds. Genome analysis can take much longer and may require multiple CPU cores. The API should enqueue a job, return a UUID, and let the UI poll or subscribe to job status.

10. Sequence preprocessing and QC

Create deterministic QC before feature extraction. QC must not mutate raw FASTA files. Record ambiguous bases, number of contigs, sequence length, GC percentage, duplicate/near-duplicate indicators, and whether the sample passes the challenge quality threshold.

```
from Bio import SeqIO
from pathlib import Path

def qc_fasta(path: str):
    total=0; contigs=0; gc=0; atgc=0; n=0
    for rec in SeqIO.parse(path, 'fasta'):
        seq=str(rec.seq).upper()
        contigs += 1; total += len(seq)
        n += seq.count('N')
        gc += seq.count('G') + seq.count('C')
        atgc += sum(seq.count(b) for b in 'ATGC')
    return {
        'length': total, 'contigs': contigs, 'ambiguous_n': n,
        'gc_fraction': (gc/atgc if atgc else None)
    }
```

Streaming principle

Your original Genome Firewall architecture already used zero-copy/memory-mapped sequence ingestion. Keep that design as the high-performance path. For the hackathon, Python can implement the reference path while a C++ scanner is compiled as an optional accelerated backend. `fileciteÖturn0file0ÖL59-L64Ö`

Optional C++ service boundary

```
// C++17 reference shape
struct QCResult {
    uint64_t bases;
    uint64_t contigs;
    uint64_t ambiguous;
    double gc_fraction;
};
```

```
QCResult scan_fasta_mmap(const char* path);
```

Keep the C++ interface narrow. Expose stable primitives such as `scan_fasta()`, `make_kmers()`, `compare_sequences()`, and `stream_features()`. Do not expose the entire internal memory model to Python.

11. DNA feature engineering: baseline

The recommended baseline in the challenge brief is one regularized logistic-regression model per antibiotic using features derived from AMRFinderPlus. That is a strong engineering choice because it is CPU-friendly, interpretable and easy to calibrate.

Feature vector

For each genome, construct a fixed feature vector. Start with binary indicators for resistance-associated genes and mutations detected in the pinned AMRFinderPlus output. Add a small number of general sequence QC features only if they improve held-out performance without becoming shortcuts.

```
X = [
  has_blaTEM_1b,
  has_blaNDM_1,
  has_tetM,
  has_ermB,
  has_gyrA_D87G,
  has_parC_S80I,
  gc_fraction,
  genome_length_log,
  contig_count_log,
  ...
]
y = [susceptible/resistant or challenge-defined label]
```

Do not use predicted phenotype as the label

AMRFinderPlus is the feature extractor; the laboratory phenotype/MIC record is the label. This keeps the learning target independent of the annotation tool. BV-BRC explicitly exposes computational method/provenance to distinguish computed AMR records from phenotype evidence. [3]

Optional raw DNA features

For a second baseline, hash canonical k-mers into a fixed-dimensional sparse vector. This allows a direct DNA-based model without storing every possible k-mer as a column.

```
# conceptual hashed k-mer feature map
index = hash(canonical_kmer) % D
X[i, index] += 1
# normalize per genome; D could be 2^18 to 2^22 for a prototype
```

12. Labels and target design

Start with a single antibiotic with sufficient laboratory observations and a clean label distribution. Expand to one model per antibiotic only after the pipeline is stable.

Target	Definition	Use
Binary AMR	R=1; S=0; I excluded or separate	Fast baseline
3-class phenotype	S/I/R	Multiclass comparison
MIC regression	log2 MIC or censored-aware target	Stretch
Time / site stratification	Holdout groups	Generalization test

Censored MIC values

MIC observations may include inequalities such as < or >. Preserve the sign. Do not convert censored measurements to an arbitrary number without documenting the rule. For the hackathon baseline, prefer the organizer's pinned categorical phenotype if available. If you build an MIC model later, use a censored regression/survival-style treatment or explicitly evaluate the bias created by simple substitution.

Dataset balancing

```
from sklearn.model_selection import train_test_split
```

```
train_ids, test_ids = train_test_split(
    sample_ids, test_size=0.20, stratify=y, random_state=42
)
```

For a robust study, go beyond a random split: group by strain/lineage or sequence similarity cluster where practical. Randomly splitting near-duplicate isolates can inflate performance and make a model look better than it is in unseen biological contexts.

Minimum label report

- Counts by antibiotic and phenotype class.
- Counts of duplicate genome IDs / repeated tests.
- Counts removed for missing or non-laboratory evidence.
- Counts by major taxonomic group.
- Train/validation/test counts and split strategy.

13. Prevent data leakage

This is one of the most important parts of the backend. The model must never see a test genome or a feature derived from the test set during feature selection, scaling, threshold tuning or hyperparameter search.

Leakage risk	Fix
Same genome appears in train and test	Deduplicate by stable genome ID
Near-duplicate isolates across splits	Cluster by sequence similarity where feasible
Feature selection on all genomes	Fit on training split only
Threshold tuned on test	Use validation/calibration set
Updated AMRFinder database	Pin annotation version
Phenotype prediction used as label	Require laboratory evidence
Post-test normalization	Fit preprocessing on train only

Pre-split vs post-split

CORRECT

raw genomes -> dedupe -> split IDs -> fit feature transform on TRAIN -> transform VAL/TEST

INCORRECT

raw genomes -> fit transform on all -> split -> train

Split manifest

```
{
  "split_id": "umbrella-v1-seed42",
  "seed": 42,
  "strategy": "grouped_by_genome_id",
  "train": ["..."],
  "validation": ["..."],
  "test": ["..."]
}
```

Commit the split manifest to the repository if it contains only IDs, not restricted data. This makes model comparisons reproducible.

14. Train the CPU baseline

Train one regularized logistic-regression classifier per antibiotic. Use the same feature schema for every target and persist the preprocessing, feature names, label mapping, model coefficients, calibration object and training manifest.

```
import joblib, pandas as pd
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

features = pd.read_parquet('data/features/amr/v1.parquet')
target = 'Ciprofloxacin'
X = features.drop(columns=['genome_id', target])
y = features[target].astype(int)

pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scale', StandardScaler(with_mean=False)),
    ('clf', LogisticRegression(
        max_iter=2000, C=1.0, penalty='l2', class_weight='balanced',
        solver='liblinear', random_state=42))
])
pipe.fit(X_train, y_train)
joblib.dump(pipe, f'data/models/{target}/baseline.joblib')
```

Why logistic regression

- Linear and sparse-friendly; easy to train on CPU.
- Coefficients expose which features drive the decision.
- Regularization limits overfitting in a high-dimensional feature table.
- Probability outputs can be calibrated and thresholded.

Do not call the output a clinical prediction

Umbrella should label the model output as research decision support. A high score can mean “the model estimates increased likelihood under this benchmark and model version,” not that a clinician should change therapy based solely on the prototype.

15. Calibrate the model and create a trust score

The UI can show a “trust” or “confidence” signal, but do not make it a mystical number. Build it from measurable components: probability calibration, training-domain similarity, missing data, annotation completeness and model provenance.

Probability calibration

```
from sklearn.calibration import CalibratedClassifierCV

calibrated = CalibratedClassifierCV(
    estimator=base_pipeline, method='sigmoid', cv=5
)

calibrated.fit(X_train, y_train)
```

Composite confidence design

```
confidence = (
    0.55 * calibrated_probability_quality +
    0.20 * feature_completeness +
    0.15 * domain_similarity +
    0.10 * provenance_integrity
)

# weights are product design choices; validate them before claiming performance.
```

Signal	Example	Interpretation
Model probability	0.91	Estimated probability under model
Calibration error	low	Whether probabilities match outcomes
Feature coverage	98%	Whether required inputs were observed
Domain similarity	high	How similar sample is to training corpus
Provenance	pinned	All input/model versions recorded

For “trust score,” show the components instead of one unlabeled number. That is much more scientifically honest and more useful to a researcher.

16. Evaluate the baseline properly

Do not optimize for one headline accuracy number. Report metrics that reveal class imbalance and probability quality.

```
from sklearn.metrics import (
    roc_auc_score, average_precision_score,
    precision_recall_fscore_support, confusion_matrix,
    brier_score_loss
)

p = model.predict_proba(X_test)[:,1]
yhat = (p >= 0.5).astype(int)

metrics = {
    'roc_auc': roc_auc_score(y_test, p),
    'pr_auc': average_precision_score(y_test, p),
    'brier': brier_score_loss(y_test, p),
    'confusion': confusion_matrix(y_test, yhat).tolist()
}
```

Minimum evaluation sheet

Metric	Why
ROC-AUC	Ranking quality
PR-AUC	Useful when resistant cases are rare
Sensitivity / recall	Missed-resistant-risk view
Specificity	False-positive control
F1	Balance between precision/recall
Brier score	Probability calibration
Calibration curve	Visual probability quality
Confusion matrix	Concrete error counts

External robustness

If time allows, hold out an organism, lineage or collection subset as a stress test. A strong result is not “99% on random split”; it is evidence that performance survives a deliberately harder split.

17. Raw DNA model: k-mer hashing

If the challenge expects “train on DNA” beyond AMR-marker features, build a second model directly from sequence-derived k-mer counts. This is practical on CPU and does not require a GPU.

Pipeline

```
FASTA
-> canonical k-mers
-> hashing trick
-> per-genome normalization
-> sparse matrix
-> logistic regression / linear SVM
-> calibrated score
```

Implementation sketch

```
from collections import Counter
import hashlib

def bucket(kmer, D):
    return int.from_bytes(hashlib.blake2b(kmer.encode(), digest_size=8).digest(), 'little') % D

def hashed_kmers(seq, k=21, D=1<<20):
    counts = Counter()
    for i in range(len(seq)-k+1):
        kmer = seq[i:i+k]
        if 'N' in kmer: continue
        rc = reverse_complement(kmer)
        key = min(kmer, rc)
        counts[bucket(key, D)] += 1
    return counts
```

For a demonstration, use one or two k values and compare against the AMRfinder feature baseline. Do not claim that a hashed k-mer model is a whole-genome language model; it is a high-dimensional classical sequence representation.

Memory control

- Process one genome at a time.
- Write sparse batches to disk.
- Store Parquet/CSR indices, not dense NumPy matrices for thousands of genomes.
- Keep D fixed across training and inference.

18. Optional deep learning: DNABERT-2 / HyenaDNA

Use pretrained DNA models rather than pretraining a foundation model during the sprint. DNABERT-2 publishes a 117M-parameter checkpoint and uses BPE tokenization with ALiBi. HyenaDNA provides long-context single-nucleotide models up to 1 million tokens and an official downstream/fine-tuning path. [6][7]

Practical architecture

```

whole bacterial genome
  -> select annotated regions / chunks
  -> pretrained encoder
  -> mean/attention pooling
  -> sample embedding
  -> small classifier
  -> calibration
  
```

Why not feed a 5 Mb genome directly?

These models have finite context windows. The practical approach is to select biologically annotated regions or chunk the genome, compute embeddings, and aggregate them into one sample representation. The challenge brief you supplied also recommends this strategy rather than pretending a full bacterial genome fits in one forward pass. [6][7]

Fine-tuning protocol

1. Freeze encoder; train a linear classifier on embeddings.
2. Evaluate against AMRFinder baseline.
3. If GPU budget allows, unfreeze only a small number of upper layers.
4. Use early stopping on validation PR-AUC.
5. Never tune on the test set.

On a single-GPU hackathon build, the safest value is “pretrained embeddings + lightweight classifier.” Full pretraining on the 120 GB corpus is a post-hackathon experiment.

19. Model registry and reproducibility

Every deployed model should have a manifest. This is especially important because AMRFinderPlus databases and external pretrained models change over time.

```
{
  "model_id": "ciprofloxacin-baseline-001",
  "target": "Ciprofloxacin",
  "model_type": "logistic_regression",
  "feature_schema": "amr-v1",
  "amrfinder_software": "PINNED_VERSION",
  "amrfinder_database": "YYYY-MM-DD.N",
  "training_dataset": "challenge-v1",
  "split_manifest": "umbrella-v1-seed42",
  "python": "3.x",
  "packages_lock_hash": "...",
  "created_at": "2026-10-03T...Z",
  "metrics": {"roc_auc": 0.0, "pr_auc": 0.0}
}
```

Database tables

```
CREATE TABLE model_registry (
  model_id TEXT PRIMARY KEY,
  target TEXT NOT NULL,
  model_type TEXT NOT NULL,
  feature_schema TEXT NOT NULL,
  artifact_uri TEXT NOT NULL,
  training_manifest JSONB NOT NULL,
  metrics JSONB NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  active BOOLEAN NOT NULL DEFAULT false
);
```

Artifact naming

```
models/<target>/<model_id>/
model.joblib
schema.json
metrics.json
calibration.json
manifest.json
feature_names.txt
```

20. FastAPI backend

Keep the API resource-oriented. Apps should never call Python scripts or shell commands directly.

```
from fastapi import FastAPI, UploadFile, File
from pydantic import BaseModel

app = FastAPI(title='Umbrella OS Science API', version='1.0.0')

class AnalysisRequest(BaseModel):
    sample_id: str
    workflow: str
    model_id: str | None = None

@app.post('/v1/samples')
async def create_sample(file: UploadFile = File(...)):
    ...

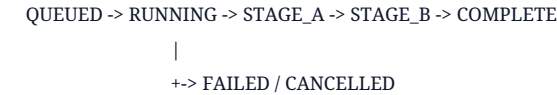
@app.post('/v1/jobs')
async def create_job(req: AnalysisRequest):
    ...

@app.get('/v1/jobs/{job_id}')
async def get_job(job_id: str):
    ...
```

Required endpoints

Endpoint	Purpose
POST /v1/samples	Upload FASTA/FASTQ or register object URI
GET /v1/samples/{id}	Sample metadata/QC
POST /v1/jobs	Start workflow
GET /v1/jobs/{id}	Job state + progress
GET /v1/results/{id}	Analysis result
GET /v1/amr/{sample_id}	AMRFinder findings
GET /v1/mutations/{sample_id}	Sequence comparisons
POST /v1/predict	Run trained model
GET /v1/models	Model registry
POST /v1/simulations	Start simulation
GET /v1/audit/{id}	Provenance record

Job states



21. Database schema

Use PostgreSQL for metadata and results, not raw multi-GB sequence blobs. Put raw FASTA/FASTQ and generated artifacts in object storage.

```
CREATE TABLE samples (
  sample_id UUID PRIMARY KEY,
  genome_id TEXT,
  organism TEXT,
  sequence_uri TEXT NOT NULL,
  sequence_sha256 TEXT NOT NULL,
  qc JSONB,
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE amr_findings (
  id BIGSERIAL PRIMARY KEY,
  sample_id UUID REFERENCES samples(sample_id),
  gene TEXT,
  mutation TEXT,
  class_name TEXT,
  method TEXT NOT NULL,
  software_version TEXT NOT NULL,
  database_version TEXT NOT NULL,
  raw_artifact_uri TEXT
);

CREATE TABLE analysis_jobs (
  job_id UUID PRIMARY KEY,
  sample_id UUID,
  workflow TEXT NOT NULL,
  state TEXT NOT NULL,
  progress REAL DEFAULT 0,
  artifact_uri TEXT,
  error TEXT,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);
```

Vector search

Use pgvector only for text/document retrieval or scientific note embeddings. Do not pretend a vector database is the correct primary store for genomic sequence. Keep sequences in files/object storage and index them with purpose-built sequence/k-mer structures where needed.

22. Object storage + job queue

For local development use MinIO. For deployment use S3. Store immutable source files and derived artifacts with content-addressable names where practical.

```
s3://umbrella-data/
  raw/<sha256>/<filename>
  amrfinder/<db_version>/<sample_id>.tsv
  qc/<sample_id>.json
  features/<feature_schema>/<split>.parquet
  models/<target>/<model_id>/...
  reports/<run_id>/...
```

Redis/Celery

```
from celery import Celery

celery = Celery('umbrella', broker='redis://localhost:6379/0')

@celery.task(bind=True)
def analyze_sample(self, sample_id):
    self.update_state(state='STAGE_A', meta={'progress': 20})
    # QC
    self.update_state(state='STAGE_B', meta={'progress': 55})
    # AMRFinderPlus
    self.update_state(state='MODEL', meta={'progress': 80})
    # model inference
    self.update_state(state='COMPLETE', meta={'progress': 100})
```

Failure strategy

- Retry network downloads with exponential backoff.
- Do not retry deterministic input-validation failures.
- Record stderr and command version for scientific jobs.
- Make every stage idempotent using input hashes and stage manifests.

23. Genome Analyzer backend

Genome Analyzer is the read-only gateway to raw sequence statistics and annotations. It should be fast because the expensive computation happened asynchronously.

Field	Source / computation
Length	FASTA parser
Contigs	FASTA parser
GC%	FASTA parser
N content	FASTA parser
AMR markers	AMRFinderPlus artifact
Genes / features	BV-BRC GFF or feature file
Taxonomy	BV-BRC metadata
Processing time	job telemetry

Endpoint

GET /v1/samples/{sample_id}

```
{
  "sample_id": "...",
  "organism": "...",
  "qc": {"length": 5210000, "gc": 0.51},
  "annotation": {
    "amrfinder_db": "...",
    "findings": 7
  }
}
```

UI rule

The desktop should show the sample as a scientific object. Opening the same sample from Sample Vault, Genome Analyzer, Mutation Lab or AMR Sentinel must preserve context and never create separate copies of the underlying data.

24. Mutation Lab backend

Mutation Lab compares a reference and query sequence computationally. The MVP can support SNP-like single-base substitutions, insertions/deletions where alignment is available, and region-level differences. It should provide coordinates and evidence, not experimental instructions.

Two implementation paths

Path	Implementation
Fast MVP	Simple pairwise comparison for same-length regions
Advanced	Use a mature aligner/library in a worker container
Large-scale	Use k-mer/sequence index such as XTree for lookup/coverage workflows

XTree is a k-mer-based sequence lookup/alignment tool designed for efficient large reference collections, with multithreading and pre-built database support. It is useful as a scalable optional backend for the Pathogen Atlas rather than as the core model itself. [8]

Result contract

```
{
  "reference": "sample-A",
  "query": "sample-B",
  "variants": [
    {"position": 18321, "type": "SNP", "ref": "A", "alt": "G"}
  ],
  "summary": {
    "snps": 14, "insertions": 1, "deletions": 2
  },
  "provenance": {"aligner": "...", "version": "..."}
}
```

25. AMR Sentinel backend

AMR Sentinel is the evidence-first interface to resistance-associated findings. It should merge AMRFinderPlus output with laboratory phenotype only at the analysis layer; the two sources remain distinct in storage.

```
GET /v1/amr/{sample_id}?antibiotic=Ciprofloxacin
```

```
{
  "findings": [
    {
      "feature": "gyrA_D87G",
      "type": "point_mutation",
      "source": "AMRFinderPlus",
      "database_version": "..."
    }
  ],
  "prediction": {
    "model_id": "ciprofloxacin-baseline-001",
    "probability": 0.91,
    "confidence": 0.84
  }
}
```

Evidence panel

- Observed genomic feature(s).
- Annotation source + version.
- Laboratory phenotype rows for the same sample/genome, when available.
- Model version and validation metrics.
- Domain similarity / out-of-distribution warning if supported.

The result should explicitly distinguish: “observed marker,” “laboratory result,” and “model estimate.” This prevents the UI from collapsing different evidence types into one misleading score.

Radiation model

Input state

- organism_model
- stress_class
- relative_intensity
- duration_bucket

Simulation state

- DNA_integrity
- oxidative_stress
- repair_load
- mutation_signal

Output

- damage_profile
- uncertainty
- explanatory_notes

ASCII visualization

```

      -----
    /  CELL   \
   |  5DNA5   |
   |  55ROS55 ---> |
   |  ^       |
  \| \| \| /
    -----

```

Treat these values as outputs of a simulation model, not experimentally measured biological effects. The app should show model assumptions and an uncertainty panel.

27. Vella orchestration + App Forge backend

Vella is the natural-language control plane. It translates an intent into one or more typed backend actions. App Forge creates a new application from a constrained specification instead of executing arbitrary generated code inside the OS.

User: "build a genome comparator"

|

v

Intent parser

|

v

App spec

- screens

- data contracts

- actions

- permissions

|

v

Validator

|

v

Runtime package

|

v

Install into App Registry

App manifest

```
{
  "name": "Genome Comparator",
  "version": "1.0.0",
  "permissions": ["samples:read", "analysis:execute"],
  "routes": ["/compare"],
  "capabilities": [
    "sequence.compare",
    "report.export"
  ]
}
```

Minimum Forge backend

- POST /v1/apps/generate - create draft spec.
- POST /v1/apps/{id}/validate - validate schema/permissions.
- POST /v1/apps/{id}/install - publish to registry.
- PATCH /v1/apps/{id} - evolve an installed app.

28. Kiro-driven build process

The Kiro workflow should be visible in your engineering artifacts. Use Specs for requirements and task decomposition, Steering for architecture/security rules, and Hooks for repeatable validation. Kiro supports spec-driven development, custom agents, hooks and MCP integrations. [9][10]

Suggested Kiro files

```
.kiro/  
  specs/  
    umbrella-kernel.md  
    data-pipeline.md  
    amr-sentinel.md  
    model-training.md  
    app-forge.md  
  steering/  
    architecture.md  
    security.md  
    science-provenance.md  
    coding-standards.md  
  hooks/  
    test-backend.json  
    validate-model-manifest.json  
    lint-generated-app.json
```

Agent roles

Agent	Job
Architect	turn requirements into modules/contracts
Data Engineer	ETL, manifests, Parquet, schema
Bioinformatics Engineer	FASTA/AMRFinder pipeline
ML Engineer	features, training, evaluation
Security Reviewer	permissions, sandboxing, secrets
QA Engineer	integration and reproducibility tests

Kiro hook examples

- On backend file change: run unit tests.
- On model manifest change: validate schema and metrics JSON.
- On generated app change: lint + typecheck + permission validation.
- On pipeline config change: verify dataset version and input hashes.

29. Final 18.5-hour execution checklist

Window	Non-negotiable deliverable
0:00-1:00	Repo, WSL, Docker, env, DB, Redis, object store
1:00-2:30	Organizer dataset manifest + small sample loaded
2:30-4:00	FastAPI upload/sample/job endpoints
4:00-6:30	FASTA QC + AMRFinderPlus worker
6:30-9:00	AMR normalization + training Parquet
9:00-11:30	Logistic baseline for 1-3 antibiotics
11:30-13:00	Calibration + metrics + model registry
13:00-15:00	Genome / Mutation / AMR / Analysis apps
15:00-16:30	Vella orchestration + App Forge demo
16:30-17:30	Demo dataset + logs + polished UI
17:30-18:00	End-to-end smoke test
18:00-18:30	README, architecture diagram, Kiro evidence

Post-hackathon training plan

- Expand to all organizer-pinned genomes and antibiotics.
- Add grouped/clustered evaluation and external holdouts.
- Add raw DNA k-mer model and compare with marker baseline.
- Add pretrained DNABERT-2 or HyenaDNA embeddings on selected regions.
- Add XTree-backed reference search for very large sequence collections.
- Move object storage and job execution to AWS S3 + ECS/EC2 + SQS + RDS when the platform needs scale.

Source references

- [1] BV-BRC Documentation and FTP/CLI guides: <https://www.bv-brc.org/docs/> and https://www.bv-brc.org/docs/quick_references/ftp.html
- [2] NCBI AMRFinderPlus: <https://www.ncbi.nlm.nih.gov/pathogens/antimicrobial-resistance/AMRFinder/>
- [3] BV-BRC genome_amr API: https://www.bv-brc.org/api/doc/genome_amr
- [4] BV-BRC FTP: https://www.bv-brc.org/docs/quick_references/ftp.html
- [5] NCBI AMRFinderPlus source/update implementation: <https://github.com/ncbi/amr>
- [6] DNABERT-2: https://github.com/MAGICS-LAB/DNABERT_2
- [7] HyenaDNA: <https://github.com/HazyResearch/hyena-dna>
- [8] XTree: <https://github.com/two-frontiers-project/2FP-XTree>
- [9] Kiro documentation: <https://kiro.dev/docs/>
- [10] Kiro hooks/custom agents: <https://kiro.dev/docs/hooks/> and <https://kiro.dev/docs/custom-agents/>

Safety note: Umbrella OS is a computational research platform. Biological simulation features should be framed as model-based analysis/education, and the system should not generate actionable wet-lab pathogen modification or optimization procedures.